COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/16/26	**DATE SUBMITTED:**	07/16/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Quejado, Jimlord

ACTIVITY NO. 2.1 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

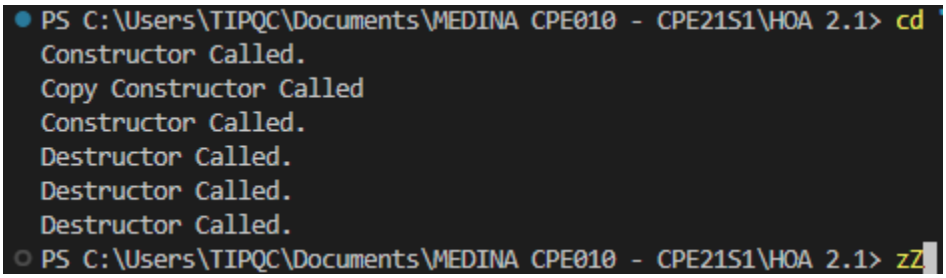
Screenshot	
Observation	Based on the output, the code that was run shows the successful execution of our constructor, deconstructor, and copy constructor defined in the "**Student**" class of the code. It first started with **student1** being used to create the first object, overwriting the default parameters of the constructor by providing a name, '*Roman*' and an age, *28*. The copy constructor is then called as the object for **student2** is initialized from our already existing **student1**. Another constructor is called as the object, **student3** is created with default parameters, '*John Doe*' and age *18*. Right after, since a copy assignment operator isn't defined within our class, the compiler automatically generates one as **student3** is assigned the same data as **student2**. Then, as the program ends, the destructors are automatically called to delete the previously created objects, where the program finally ends its execution.

Table 2-1. Initial Driver Program

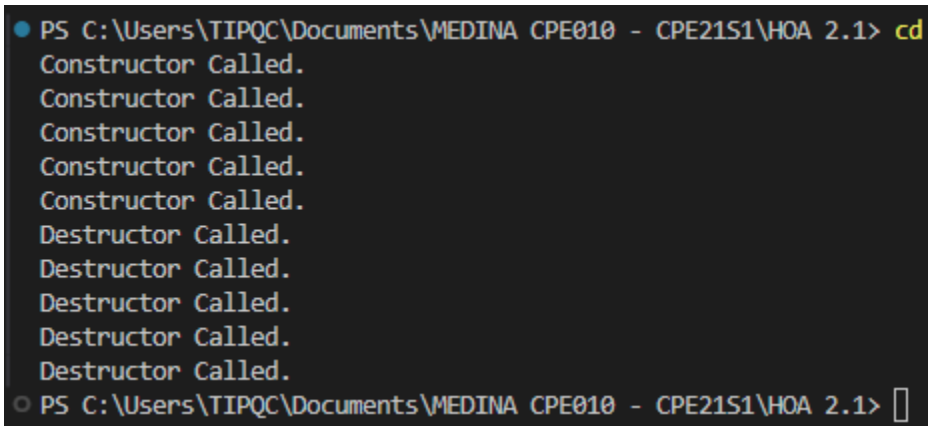
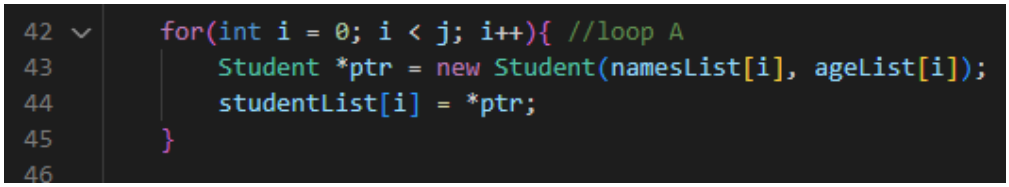
Screenshot	
Observation	The execution of the code in this segment is far simpler than with the previous part. The first five constructors are called to create five objects, **studentList**, given by the constant that was declared and initialized with the value of 5. Despite creating two different string and integer arrays containing different names and ages (**namesList** & **ageList**), these aren't actually connected in any way to the initially created objects in **studentList**. So we'd end up having 5 objects with '*John Doe*' all with ages 18, a separate string array with different names under **namesList** and a separate integer array with different numbers tilted **ageList**. At the end of the execution, the destructor is automatically called once more, deleting the five previously created objects and hence why it appears five times as the program finishes.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	This part of the code uses a for loop in order to dynamically allocate 5 student objects on the heap (using the constant variable, **size_t** initialized with the mentioned size). Right after, they are assigned individually with each corresponding element in **namesList** and **ageList** which successfully changes the created objects from '*John Doe*' all aged with 18 to '*Carly, Freddy, Sam, Zack, and Cody*' all with their respective ages. A problem that arises from this loop though is that ptr is never deallocated and remains locked up until end of the execution, a 'memory leak' is created here.

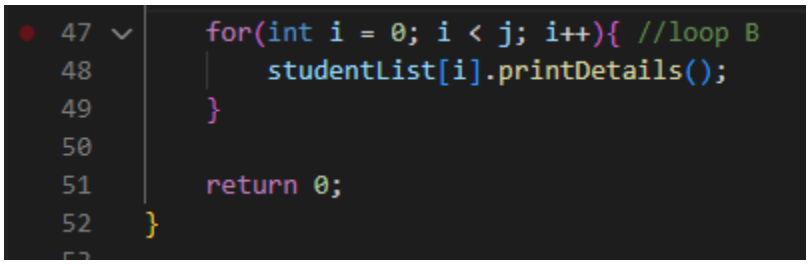
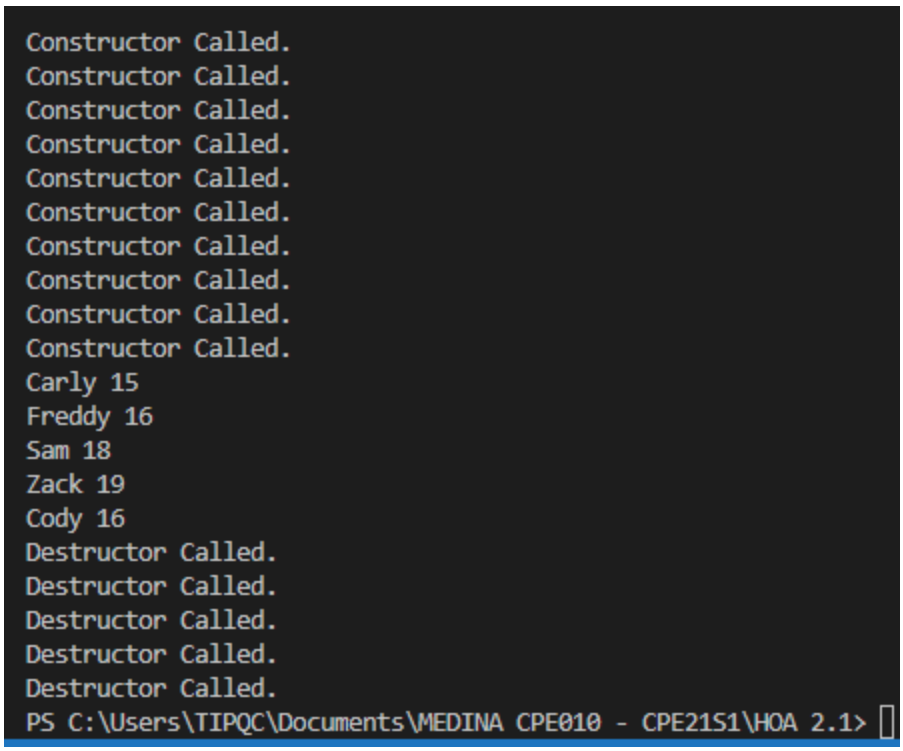
Loop B	
Observation	Loop B just allows for the printing of the created objects with proper formatting with the use of a for-loop, calling back on the **printDetails** function in the original **Student** class.
Output	
Observation	For the entirety of the output, the flow of the program goes as follows. 5 default students are created, followed by another 5 assigned students under loop A on the heap which make up the 10 constructor calls in the output. The copy constructor isn't called at all and loop B proceeds to print out the student details. In loop A, the **new** keyword is used but no partnering **delete** which causes a memory leak. The destructors for it aren't called such as for with the initial studentList. Hence, only five destructors are called in the end.

Table 2-3. Final Driver Program

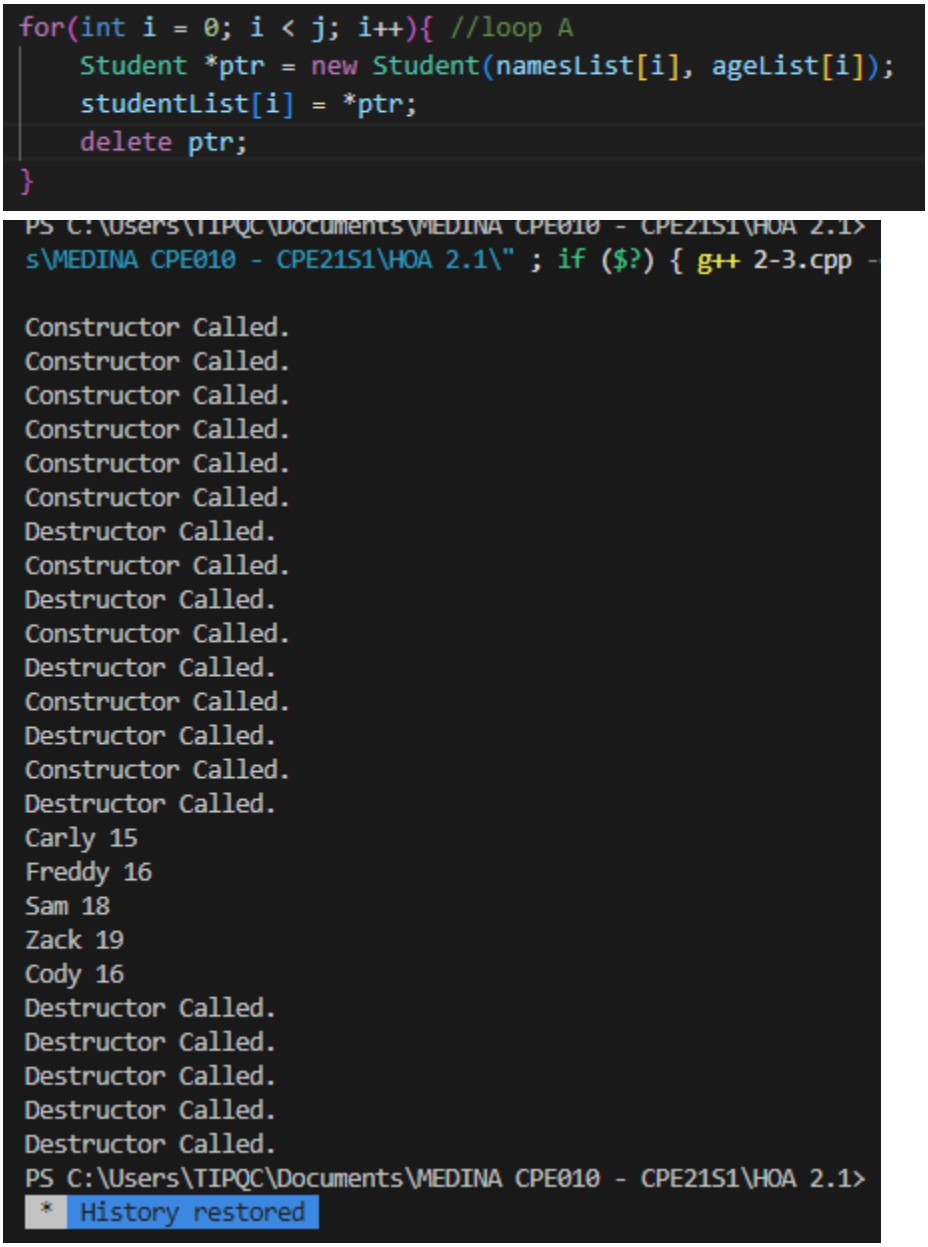
Modifications	
Observation	The only modification I added was within loop A, adding the needed 'delete ptr;' to successfully deallocate the memory after use. This follows 'AUDNU!' where we allocate, use, and finally delete with the exception of not needing to use a '*nullptr*' because the variable doesn't need to be used again before it disappears as the loop ends (which also just deletes the pointer anyways).

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers

Jenny's Grocery List		
Apple	PHP 10	x7
Banana	PHP 10	x8
Broccoli	PHP 60	x12
Lettuce	PHP 50	x10

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

Problem 5: Discuss.

Output:

supplementary.cpp > foodType > findSum()

```cpp
#include <iostream>
#include <string>

// base class for fruit and vegetable to inherit from
class foodType {
    protected:
        std::string name;
        int price;
        int quantity;

    public:
        foodType(std::string giveName, int givePrice, int giveAmount) { // constructor!
            name = giveName;
            price = givePrice;
            quantity = giveAmount;
        }
```

```cpp
    virtual ~foodType() {} // dynamic destructor for newly created pointer to the created object

    virtual void showContents() = 0;

    int findSum() { // total price when we have more than one to buy
        return price * quantity;
    }

    std::string getName() { // getter function to give name
        return name;
    }
};



// Inheritence (fruit and veggie classes - PROBLEM 1)

class Fruit : public foodType {

public:
    // Constructor
    Fruit(std::string giveName = "Null Fruit", int givePrice = 0, int giveAmount = 0) // default
    : foodType(giveName, givePrice, giveAmount) { // this line calls the base class constructor first
    std::cout << "Fruit Constructor Called: " << name << std::endl;
    }

    // Destructor
    ~Fruit() override {
        std::cout << "Fruit Destructor Called: " << name << std::endl;
    }

    // Copy Constructor
    Fruit(const Fruit &copyFruit) : foodType(copyFruit.name, copyFruit.price, copyFruit.quantity) {
        std::cout << "Fruit Copy Constructor Called: " << name << std::endl;
    }

    // Copy Assignmnent Operator
    Fruit& operator=(const Fruit &assignFruit) {
        std::cout << "Fruit Copy Assignment Operator Called for: " << assignFruit.name << std::endl;
        if (this != &assignFruit) { // check if self-assignment and skips the unnesscesarry-ness
            name = assignFruit.name;
            price = assignFruit.price;
            quantity = assignFruit.quantity;
        }
        return *this;
    }

    void showContents() override {
        std::cout << "[Fruit] " << name << " || Price: PHP " << price << " || Qty: x" << quantity << " || Subtotal: PHP " << findSum() << std::endl;
    }
};

class Vegetable : public foodType { // I just copy and pasted my fruit class and just changed the fruits in here
public:
    // Constructor
    Vegetable(std::string newName = "Null Veggie", int newPrice = 0, int newQuantity = 0)
        : foodType(newName, newPrice, newQuantity) {
        std::cout << "Vegetable Constructor Called: " << name << std::endl;
    }
```

```cpp

    // Destructor
    ~Vegetable() override {
        std::cout << "Vegetable Destructor Called: " << name << std::endl;
    }

    // Copy Constructor
    Vegetable(const Vegetable &copyVeg) : foodType(copyVeg.name, copyVeg.price, copyVeg.quantity) {
        std::cout << "Vegetable Copy Constructor Called: " << name << std::endl;
    }

    // Copy Assignment Operator
    Vegetable& operator=(const Vegetable &assignVeg) {
        std::cout << "Vegetable Copy Assignment Operator Called: " << assignVeg.name << std::endl;
        if (this != &assignVeg) {
            name = assignVeg.name;
            price = assignVeg.price;
            quantity = assignVeg.quantity;
        }
        return *this;
    }

    void showContents() override {
        std::cout << "[Vegetable] " << name << " || Price: PHP " << price << " || Qty: x" << quantity << " || Subtotal: PHP " << findSum() << std::endl;
    }
};

// Function prototype/s
void totalPrice(foodType* list[], int size);


// MAIN
int main() {
    std::cout << "Jenna's Grocery List\n";
    // creating the array
    const int size = 4;
    foodType* itemList[size];
```

```
114
115     itemList[0] = new Fruit("Apple", 10, 7);
116     itemList[1] = new Fruit("Banana", 10, 8);
117     itemList[2] = new Vegetable("Broccoli", 60, 12);
118     itemList[3] = new Vegetable("Lettuce", 50, 10);
119
120     // using a for-loop to display the items
121     std::cout << "\nItems: \n";
122     for (int i = 0; i < size; i++) {
123         if (itemList[i] != nullptr) {
124             itemList[i]->showContents();
125         }
126     }
127
128     // function call from problem 3
129     totalPrice(itemList, size);
130
131     // delete lettuce and destroy it (PROBLEM 4)
132     std::cout << "\nRemoving lettuce...\n";
133     for (int i = 0; i < size; i++) {
134         if (itemList[i] != nullptr && itemList[i]->getName() == "Lettuce") {
135             delete itemList[i];     // D
136             itemList[i] = nullptr;  // NU!
137             std::cout << "Lettuce is gone, reduced to atoms.\n";
138         }
139     }
140
141     // check after lettuce destruction
142     std::cout << std::endl; // spacing
143     totalPrice(itemList, size);
144
145     // finally, deallocating
146     std::cout << "\nCleaning up. Thank you for your time.\n";
147     for (int i = 0; i < size; i++) {
148         if (itemList[i] != nullptr) {
149             delete itemList[i];
150         }
151     }
152
153     return 0;
154 }
```

```cpp
// Function definition/s (PROBLEM 3)
void totalPrice(foodType* list[], int size) { // basically, accepts an array of pointers to our base class, foodType and number of elements in the array
    int total = 0; // start at 0 alwayss
    for (int i = 0; i < size; i++) { // use a for loop to cycle through the array for every pointer present
        if (list[i] != nullptr) { // check if its actually a pointer to an actual object (prevents crashes, learned the hard way)
            total += list[i]->findSum(); // makes sure quantity comes into play
        }
    }
    std::cout << "Total Price: PHP " << total << std::endl;
}
```

Execution Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Vegetable Destructor Called: Broccoli
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 2.1> cd "c:\Users
Jenna's Grocery List
Fruit Constructor Called: Apple
Fruit Constructor Called: Banana
Vegetable Constructor Called: Broccoli
Vegetable Constructor Called: Lettuce

Items:
[Fruit] Apple || Price: PHP 10 || Qty: x7 || Subtotal: PHP 70
[Fruit] Banana || Price: PHP 10 || Qty: x8 || Subtotal: PHP 80
[Vegetable] Broccoli || Price: PHP 60 || Qty: x12 || Subtotal: PHP 720
[Vegetable] Lettuce || Price: PHP 50 || Qty: x10 || Subtotal: PHP 500
Total Price: PHP 1370

Removing lettuce...
Vegetable Destructor Called: Lettuce
Lettuce is gone, reduced to atoms.

Total Price: PHP 870

Cleaning up. Thank you for your time.
Fruit Destructor Called: Apple
Fruit Destructor Called: Banana
Vegetable Destructor Called: Broccoli
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA 2.1>
```

Code Analysis:

When I initially came across the problem, I immediately thought of a lot of ambitious features. It wanted me to create classes for fruits and vegetables, I then remembered my python CPE009A class where we learned about polymorphism which was one of my more favorite topics. So I decided to do just that and challenge myself by finding the C++ equivalent and applying it to the activity. I started with a base item where both vegetable and fruit classes inherit from. Additionally, after searching for a bit, I found the use of 'virtual' that properly cleans up newly created dynamic pointers to prevent memory leaks and to make sure the compiler sees the program 'dynamically' instead of initializing the functions right from the get go (like

seeing where the pointer goes in actual runtime instead of deciding immediately). Other than that, I just used for-loops and lists to output as well as store information, defined functions, and put them all together in the main. Overall, I can say that I'm proud of what I've done and I've refreshed as well as learned a lot about the subject.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Used in explaining how each component of the code interacts with each other for visualization and clarifying concepts I had a hard time fully grasping. No instances of code creation were asked even once for this lab activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.